



CHRONIQO

RISIKOANALYSE

SW7BAC-01 BACHELORPROJEKT

BILAG 3

Elev:

Mads Dittmann Villadsen

202209869 / AU715368

Vejleder:

Tommy Bjerre Nielsen

tbn@ece.au.dk

MAJ 2026

Indholdsfortegnelse

Figurliste	ii
Tabelliste	iii
1. Indledning	1
2. Risici og vurdering.....	2
2.1. Oversigt over identificerede risici.....	3
2.2. Risikomatrix.....	5
2.3. Risikokategorisering	6
2.3.1. Høj-prioritetsrisici (impact ≥ 15)	6
2.3.2. Mellem-prioritetsrisici (impact 9-14).....	7
2.3.3. Lav-prioritetsrisici (impact < 9).....	8
3. Risikohåndtering	9
3.1. Mitigation-planer	10
3.1.1. R1: Personlige helbredsudfordringer (ryg og/eller fatigue).....	10
3.1.2. R13: Sikkerhedshuller (OAuth 2.0-implementation).....	10
3.1.3. R14: Problemer relateret til systemets deployment	10
3.1.4. R9: Afhængighed af eksterne tjenester (f.eks. Auth Provider)	11
3.2. Contingency-planer.....	12
3.2.1. R1: Personlige helbredsudfordringer (ryg / fatigue).....	12
3.2.2. R13: Sikkerhedshuller (OAuth 2.0-implementation).....	12
3.2.3. R14: Problemer relateret til systemets deployment	12
3.2.4. R9: Afhængighed af eksterne tjenester (f.eks. Auth Provider)	13
4. Samlet konklusion.....	14
5. Referencer	15
6. Bilag.....	16

Figurliste

Figur 1: Grafisk repræsentation af projektets risikovurdering.....	5
--	---

Tabelliste

Tabel 1: Tabeloversigt over de identificerede risici	3
--	---

1. Indledning

Dette dokument præsenterer risikoanalysen for Chroniqo-projektet. Formålet er at identificere, kvantificere og prioritere de risici, der kan true projektets gennemførelse eller kvaliteten af det færdige system, og på baggrund heraf udlede konkrete håndteringsstrategier.

Analysen er udarbejdet tidligt i projektforsløbet og har haft direkte indflydelse på både teknologivalg og udviklingsmetode. Eksempelvis er valget af et fullstack-monorepo med høj typesikkerhed og etablerede, veldokumenterede teknologier delvist begrundet i ønsket om at reducere den tekniske risiko under et enkeltmandsprojekt med særlige kapacitetsbetingelser - illustreret ved R1 (Personlige helbredsudfordringer), som er vurderet til den højeste impact-score i analysen.

Risikovurderingen følger en kvalitativ metode, hvor hver risiko scores på en skala fra 1 til 5 for henholdsvis sandsynlighed og konsekvens. Impact-scoren beregnes som produktet af de to (Sandsynlighed x Konsekvens) og bruges til at prioritere indsatsen i risikohåndteringen. For høj-prioritetsrisici ($\text{impact} \geq 15$) er der udarbejdet både mitigation-planer (forebyggende tiltag) og contingency-planer (reaktive tiltag, såfremt risikoen bliver til virkelighed).

2. Risici og vurdering

I dette afsnit præsenteres den indledende risikovurdering for projektet. Formålet er at identificere og vurdere potentielle risici, der kan påvirke projektets succes, samt danne grundlag for risikohåndteringsstrategier. Vurderingen er baseret på en kvalitativ analyse af sandsynlighed og konsekvens.

2.1. Oversigt over identificerede risici

Følgende Tabel 1 giver et samlet overblik over de identificerede risici, deres sandsynlighed, konsekvens og den beregnede impact-score (Sandsynlighed x Konsekvens). Dette overblik danner grundlag for den detaljerede analyse og prioritering i de efterfølgende underafsnit.

Tabel 1: Tabeloversigt over de identificerede risici, deres sandsynlighed, konsekvens og den beregnede impact-score (Sandsynlighed x Konsekvens).

Oversigt over identificerede risici				
ID	Titel	Sandsynlighed (1-5)	Konsekvens (1-5)	Impact
R1	Personlige helbredsudfordringer (ryg og/eller fatigue)	5	5	$5 \times 5 = 25$
R2	Scope Creep	3	4	$3 \times 4 = 12$
R3	Estimation bias / tidsoptimisme	3	4	$3 \times 4 = 12$
R4	Fejltolkning eller ændring af krav	3	4	$3 \times 4 = 12$
R5	Utilstrækkelig testdækning	3	4	$3 \times 4 = 12$
R6	Tab af adgang til repository / versions-styringsplatform	1	5	$1 \times 5 = 5$
R7	Dataintegritetsfejl (tab eller inkonsistens i databasedata)	2	5	$2 \times 5 = 10$
R8	Forkert håndtering af personfølsomme data / privacy-fejl	2	5	$2 \times 5 = 10$
R9	Afhængighed af eksterne tjenester (f.eks. Auth Provider)	3	5	$3 \times 5 = 15$
R10	Performance-problemer (langsomt feed ved mange brugere)	3	3	$3 \times 3 = 9$
R11	Teknisk kompleksitet i anvendte frameworks og biblioteker	3	3	$3 \times 3 = 9$

R12	Uhensigtsmæssige arkitektur- eller designbeslutninger	3	3	$3 \times 3 = 9$
R13	Sikkerhedshuller (OAuth 2.0-implementation)	4	5	$4 \times 5 = 20$
R14	Problemer relateret til systemets deployment	4	4	$4 \times 4 = 16$
R15	Brugerrelaterede UX-problemer	3	2	$3 \times 2 = 6$

2.2. Risikomatrix

For at skabe et struktureret overblik over projektets væsentligste usikkerheder er de identificerede risici analyseret og placeret i en 5x5 risikomatrix. Matrixen er farvekodet fra lav risiko (grøn, nederste venstre) til høj risiko (rød, øverste højre), hvilket tydeliggør de områder, der kræver mest opmærksomhed.

Figur 1 illustrerer den grafiske repræsentation af risiciene baseret på deres vurderede sandsynlighed (x-akse) og konsekvens (y-akse):

Risikovurderingsmatrix		Sandsynlighed				
		Meget usandsynlig (1)	Usandsynlig (2)	Mulig (3)	Sandsynlig (4)	Meget sandsynlig (5)
Konsekvens	Kritisk (5)	R6: Tab af adgang til repository / versionsstyringsplatform	R7: Dataintegritetsfejl (tab eller inkonsistens i databasedata) R8: Forkert håndtering af personfølsomme data / privacy-fejl	R9: Afhængighed af eksterne tjenester (fx Auth Provider)	R13: Sikkerhedshuller (OAuth 2.0-implementation)	R1: Personlige helbredsudfordringer (Ryg/Fatigue)
	Stor (4)			R2: Scope Creep R3: Estimation bias / tidsoptimisme R4: Fejltolkning eller ændring af krav R5: Utilstrækkelig testdækning	R14: Problemer relateret til systemets deployment	
	Moderat (3)			R10: Performance-problemer (langsomt feed ved mange brugere) R11: Teknisk kompleksitet i anvendte frameworks og biblioteker R12: U hensigtsmæssige arkitektur- eller designbeslutninger		
	Lille (2)			R15: Brugerrelaterede (UX)		
	Ubetydelig (1)					

Figur 1: Grafisk repræsentation af projektets risikovurdering baseret på sandsynlighed og konsekvens

2.3. Risikokategorisering

For at skabe et dybdegående overblik og lette den efterfølgende risikohåndtering er risiciene kategoriseret efter deres impact-score.

2.3.1. Høj-prioritetsrisici (impact ≥ 15)

De højt-prioriterede risici repræsenterer de mest kritiske trusler mod projektet, med potentiale for alvorlig påvirkning af tidsplan, kvalitet og gennemførsel. En detaljeret beskrivelse af hver høj-prioritetsrisiko og de strategiske overvejelser fremgår herunder.

2.3.1.1. R1: Personlige helbredsudfordringer (ryg og/eller fatigue)

Denne risiko omfatter enkeltmandsudviklerens egen personlige helbredstilstand, herunder de kroniske rygsmerter og den dertilhørende fatigue. En forværring af helbredet kan direkte påvirke evnen til at overholde tidsplanen og opretholde en ensartet arbejdsindsats, hvilket udgør en fundamental trussel mod projektets fremdrift.

2.3.1.2. R13: Sikkerhedshuller (OAuth 2.0-implementation)

Denne tekniske risiko knytter sig til potentielle sårbarheder i håndteringen af sessioner og tredjeparts-autentificering. Fejl i implementeringen, såsom mangelfuld sikring mod XSS (Cross-Site Scripting) eller CSRF (Cross-Site Request Forgery), kan føre til uautoriseret adgang til personfølsomme brugerdata. Dette vil ødelægge tilliden til systemets sikkerhed, hvilket har kritiske konsekvenser for en platform, der lever af at være et trygt frirum.

2.3.1.3. R14: Problemer relateret til systemets deployment

Denne risiko dækker over uforudsete vanskeligheder, der kan opstå under deployment af webapplikationen til produktionsmiljøet. Dette kan inkludere konflikter med serveropsætning, uventede miljøafhængigheder eller fejl under konfiguration, hvilket kan hindre en rettidig færdiggørelse af projektet.

2.3.1.4. R9: Afhængighed af eksterne tjenester (f.eks. Auth Provider)

Denne risiko vedrører utilgængelighed af Googles OAuth API. Selvom et nedbrud i API'et vil afbryde genvejen "Continue with Google", kan berørte brugere fortsat tilgå deres konti via det normale e-mail-login. Risikoen er derfor begrænset til en forringet brugeroplevelse og øget friktion, da de berørte brugere manuelt skal logge ind i stedet for at bruge den automatiserede adgang.

2.3.2. Mellem-prioritetsrisici (impact 9-14)

Disse risici har et moderat potentiale for at påvirke projektet og kræver aktiv overvågning. De kan forsinke tidsplanen eller påvirke kvaliteten, hvis de ikke håndteres proaktivt.

2.3.2.1. R2: Scope Creep

Projektets omfang udvides ud over den oprindelige plan, hvilket resulterer i overforbrug af ressourcer og forsinkelser.

2.3.2.2. R3: Estimation bias/tidsoptimisme

Urealistiske tidsestimater kan føre til overarbejde og forringelse af kvalitet.

2.3.2.3. R4: Fejltolkning eller ændring af krav

Misforståelser af brugerbehov eller ændrede krav sent i processen kan resultere i spildt arbejde.

2.3.2.4. R5: Utilstrækkelig testdækning

Ikke alle kritiske stier og edge cases testes grundigt, hvilket kan føre til øget risiko for fejl i produktionsmiljøet.

2.3.2.5. R7: Dataintegritetsfejl

Fejl i lagring eller behandling af data kan føre til inkonsistente brugerprofiler eller opslag.

2.3.2.6. R8: Forkert håndtering af personfølsomme data

Fejl i privacy-relateret kode kan føre til brud på fortrolighed.

2.3.2.7. R10: Performance-problemer

Systemet er langsomt ved mange brugere, hvilket forringer brugeroplevelsen.

2.3.2.8. R11: Teknisk kompleksitet

Vanskeligheder med at mestre nye frameworks/biblioteker kan føre til forsinkelser.

2.3.2.9. R12: U hensigtsmæssige arkitektur- eller designbeslutninger

Tidlige tekniske valg kan låse arkitekturen eller skabe begrænsninger, der gør det svært at skalere senere.

2.3.3. Lav-prioritetsrisici (impact < 9)

Disse risici har en lavere sandsynlighed eller konsekvens, men overvåges stadig for at forhindre, at de udvikler sig til større problemer.

2.3.3.1. R6: Tab af adgang til repository/versionsstyringsplatform

Midlertidig utilgængelighed af GitLab kan forsinke udviklingen.

2.3.3.2. R15: Brugerrelaterede UX-problemer

Små fejl i brugervenligheden, der kan justeres i fremtidige iterationer.

3. Risikohåndtering

I dette afsnit præsenteres strategierne for at håndtere projektets mest kritiske risici fra afsnit 2. Risikohåndteringen er opdelt i to hovedkategorier: Mitigation-planer, der har til formål at reducere sandsynligheden eller konsekvensen af en risiko, og Contingency-planer, som beskriver, hvordan der handles, hvis en eller flere risici alligevel bliver en realitet. Strategierne er formuleret specifikt for hver høj-prioritetsrisiko for at sikre en proaktiv og responsiv tilgang.

3.1. Mitigation-planer

For at håndtere de identificerede risici proaktivt, implementeres følgende mitigation-planer. Disse tiltag skal enten afværge risiciene eller reducere deres samlede impact på projektet.

3.1.1. R1: Personlige helbredsudfordringer (ryg og/eller fatigue)

Plan: Arbejdsbyrden planlægges med rigelige buffere (jf. afsnit 3.3 i projektrapporten (1)). Den daglige arbejdstid opdeles i korte, fokuserede blokke med indlagte pauser, og der anvendes konsekvent en ergonomisk arbejdsstation. Samtidig opretholdes en løbende dialog med vejleder om helbredsstatus for at sikre rettidig forventningsafstemning.

Teknisk Mitigation (*Architectural Simplification*): For at reducere den kognitive belastning og minimere risikoen for stressrelaterede fejl, simplificeres systemets arkitektur bevidst. Frem for en distribueret Poly-repo struktur med separat frontend og backend, udvikles systemet som en fullstack monolit i Next.js (App Router) (2). Dette eliminerer overhead forbundet med tværgående netværkskald, komplekse CI/CD-pipelines og opsætning af egenudviklet OAuth-sikkerhed. Ved at reducere variable faktorer beskyttes projektets fremdrift aktivt mod uforudsete helbredsmæssige dyk.

3.1.2. R13: Sikkerhedshuller (OAuth 2.0-implementation)

Plan: For at eliminere risikoen ved egenudviklet kryptografi og sessionshåndtering ("never roll your own crypto"), uddelegeres det fulde ansvar for autentificering til branchestandard-biblioteket Auth.js (NextAuth v5) (3). Biblioteket abstraherer kompleksiteten og håndterer automatisk OAuth 2.0 PKCE-flowet (4), stateless JWT-sessioner og CSRF-beskyttelse via sikre `HttpOnly`-cookies. Der ved holdes tokens væk fra klienten, og angrebsfladen for XSS minimeres markant i forhold til en egenudviklet løsning.

3.1.3. R14: Problemer relateret til systemets deployment

Plan: Deployment-processen dokumenteres grundigt, og der oprettes et staging-miljø (`development`), som ligger så tæt op ad produktionsmiljøet (`production`) som muligt. Kontinuerlig integration/kontinuerlig deployment (CI/CD) pipelines opsættes for automatisk at teste og deploy ændringer, hvilket reducerer risikoen for manuelle fejl. Der foretages desuden tidlige test-deployments på målplatformen for at fange potentielle konflikter i opsætningen, før applikationen bliver tilgængelige for brugerne.

3.1.4. R9: Afhængighed af eksterne tjenester (f.eks. Auth Provider)

Plan: Der opbygges en fallback-mekanisme, som sikrer, at brugere altid kan tilgå deres konti via e-mail og adgangskode, hvis Google OAuth skulle fejle. Der føres løbende tilsyn med tjenestens status og opdateringer af servicevilkår for at sikre rettidig reaktion på eventuelle ændringer. Samtidig gøres det tydeligt i brugerfladen, at Google-login er en valgfri service, så brugerne er bevidste om deres alternative adgangsmuligheder.

3.2. Contingency-planer

For at sikre et effektivt beredskab, hvis de forebyggende tiltag ikke er tilstrækkelige, opstilles følgende contingency-planer. Disse planer skal aktiveres, hvis en risiko alligevel bliver en realitet, for at begrænse skaden og sikre projektets fremdrift.

3.2.1. R1: Personlige helbredsudfordringer (ryg / fatigue)

Plan: Ved perioder med akut reduceret arbejdskapacitet aktiveres en strikt omprioritering baseret på MoSCoW-analysen i Bilag 1: Kravspecifikation (5). Vejleder informeres rettidigt med henblik på forventningsafstemning og eventuel justering af milepæle. Arbejdet genoptages gradvist med udgangspunkt i eksisterende dokumentation og versionsstyring, hvilket sikrer en lav kognitiv startomkostning efter fravær. Som en sidste instans udnyttes muligheden for dispensation eller online mundtlig eksamen efter aftale med studienævnet.

Teknisk Contingency (*Scoped Reduction*): For at sikre projektets gennemførelse udnyttes den valgte Next.js-arkitektur til hurtigt at deaktivere ikke-kritiske moduler (Should-have/Could-have) uden at svække systemets samlede integritet. Da systemet er udviklet som en fullstack monolit, minimeres tidsforbruget på at rekonfigurere API-endpoints eller autentificeringsflow ved scope-reduktion. Denne arkitektoniske fleksibilitet sikrer, at en stabil kernefunktionalitet kan leveres, selv ved et væsentligt reduceret antal arbejdstimer.

3.2.2. R13: Sikkerhedshuller (OAuth 2.0-implementation)

Plan: Ved identifikation af en sikkerhedssårbarhed lukkes adgangen via den pågældende OAuth 2.0-metode øjeblikkeligt. Logdata og hændelsesforløb analyseres med henblik på årsagsidentifikation. Berørte komponenter bliver opdateret med patches, hvorefter løsningen verificeres gennem målrettet test. Om nødvendigt begrænses adgangen til systemet midlertidigt.

3.2.3. R14: Problemer relateret til systemets deployment

Plan: Ved deployment-fejl rulles der tilbage til den seneste stabile version. En detaljeret fejlfinding udføres på staging-miljøet. Hvis problemet er vedvarende, overvejes en alternativ hosting-løsning eller en forenklet deployment-strategi for at overholde den planlagte deadline.

3.2.4. R9: Afhængighed af eksterne tjenester (f.eks. Auth Provider)

Plan: Ved længerevarende utilgængelighed af eksterne tjenester deaktiveres den berørte integrationsfunktionalitet midlertidigt. Brugere henvises til alternative loginmetoder. Systemets kernefunktionalitet opretholdes uafhængigt af tredjepartstjenester. Alternativer evalueres ved behov.

4. Samlet konklusion

Risikoanalysen har identificeret 15 risici fordelt på tre prioritetsniveauer. Det samlede risikobillede for Chroniqo-projektet vurderes som moderat til højt, primært betinget af projektets enkeltmandskarakter og de særlige kapacitetsbetingelser, der kendetegner det.

De fire høj-prioritetsrisici - R1 (Personlige helbredsudfordringer), R13 (Sikkerhedshuller i OAuth 2.0-implementationen), R14 (Deployment-problemer) og R9 (Afhængighed af eksterne tjenester) - udgør tilsammen de væsentligste trusler mod projektets gennemførelse og systemets integritet. R1 skiller sig særligt ud med den højeste impact-score i analysen (i alt 25), idet en forværring af helbredet potentielt kan påvirke samtlige øvrige risici ved at reducere den tilgængelige arbejdskapacitet.

Risikoanalysen har haft direkte og målbar indflydelse på projektets tekniske og metodiske beslutninger. Valget af en fullstack monolit-arkitektur i Next.js frem for en distribueret Poly-repo struktur er eksplicit begrundet i ønsket om at reducere den kognitive belastning og sikre arkitektonisk fleksibilitet ved eventuel scope-reduktion. Tilsvarende er valget af Auth.js (NextAuth v5) som autentificeringslag en direkte konsekvens af analysen af R13, der understregede risikoen ved egenudviklet kryptografi og sessionshåndtering.

Analysen har dermed ikke udelukkende fungeret som et dokumentationsredskab, men som et aktivt styringsværktøj, der har formet grundlaget for de vigtigste tekniske valg i projektet.

5. Referencer

2. Vercel. Next.js Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs>.
3. Auth.js contributors. Authjs: Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://authjs.dev/getting-started>.
4. Parecki A. Protecting Apps with PKCE [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.oauth.com/oauth2-servers/pkce/>.

6. Bilag

1. Villadsen MD. Chroniqo: Bachelorprojekt. Projektrapport. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
5. Villadsen MD. Chroniqo: Kravspecifikation. Bilag 1. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.